Positive testing: Unary operations

Test 1 — GetMovieDetail (positive)

Endpoint: `ws://localhost:8080/ws/movies/detail`

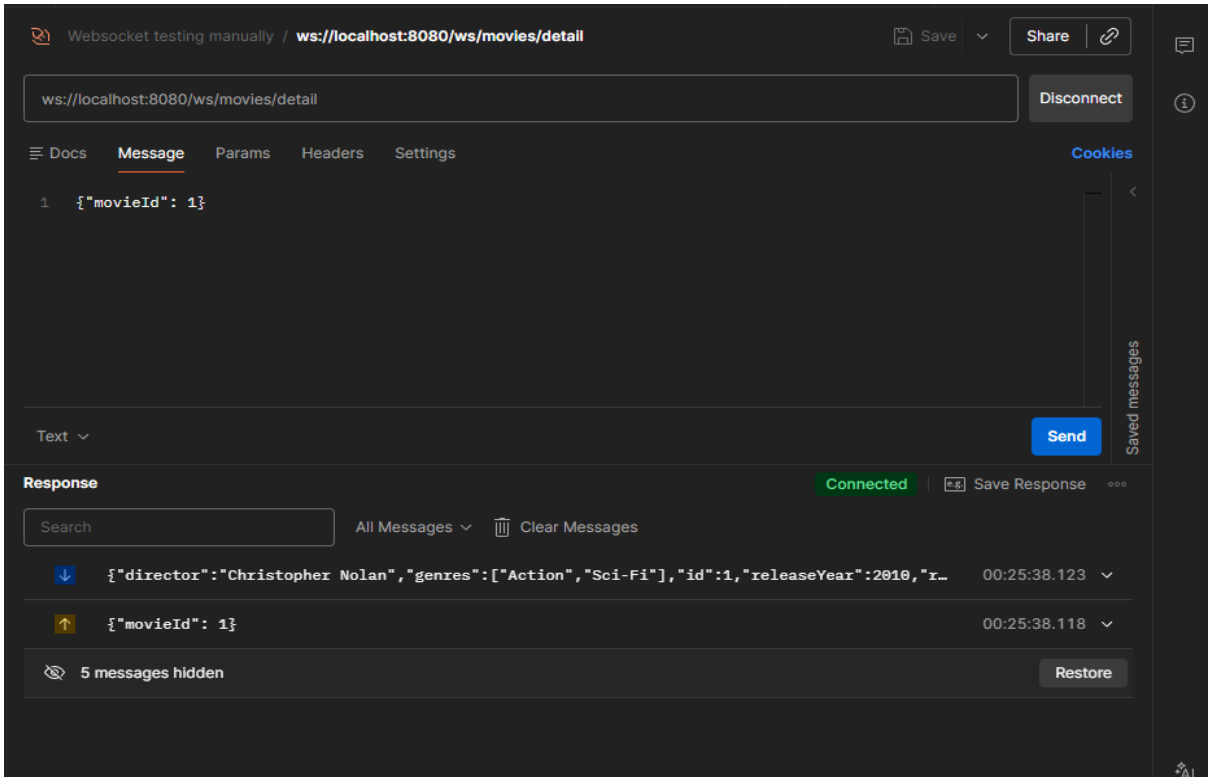
Request sent:

```
{"movieId": 1}
```

Expected response: The server returns a `MovieDetailResponse` JSON object containing the full movie details for Inception, including a `genres` array.

```
{
  "id": 1,
  "title": "Inception",
  "releaseYear": 2010,
  "runtimeMinutes": 148,
  "director": "Christopher Nolan",
  "synopsis": "...",
  "genres": ["Action", "Sci-Fi"]
}
```

Result:



Negative testing: Unary operations

Test 2 — GetMovieDetail (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/detail`

Request sent:

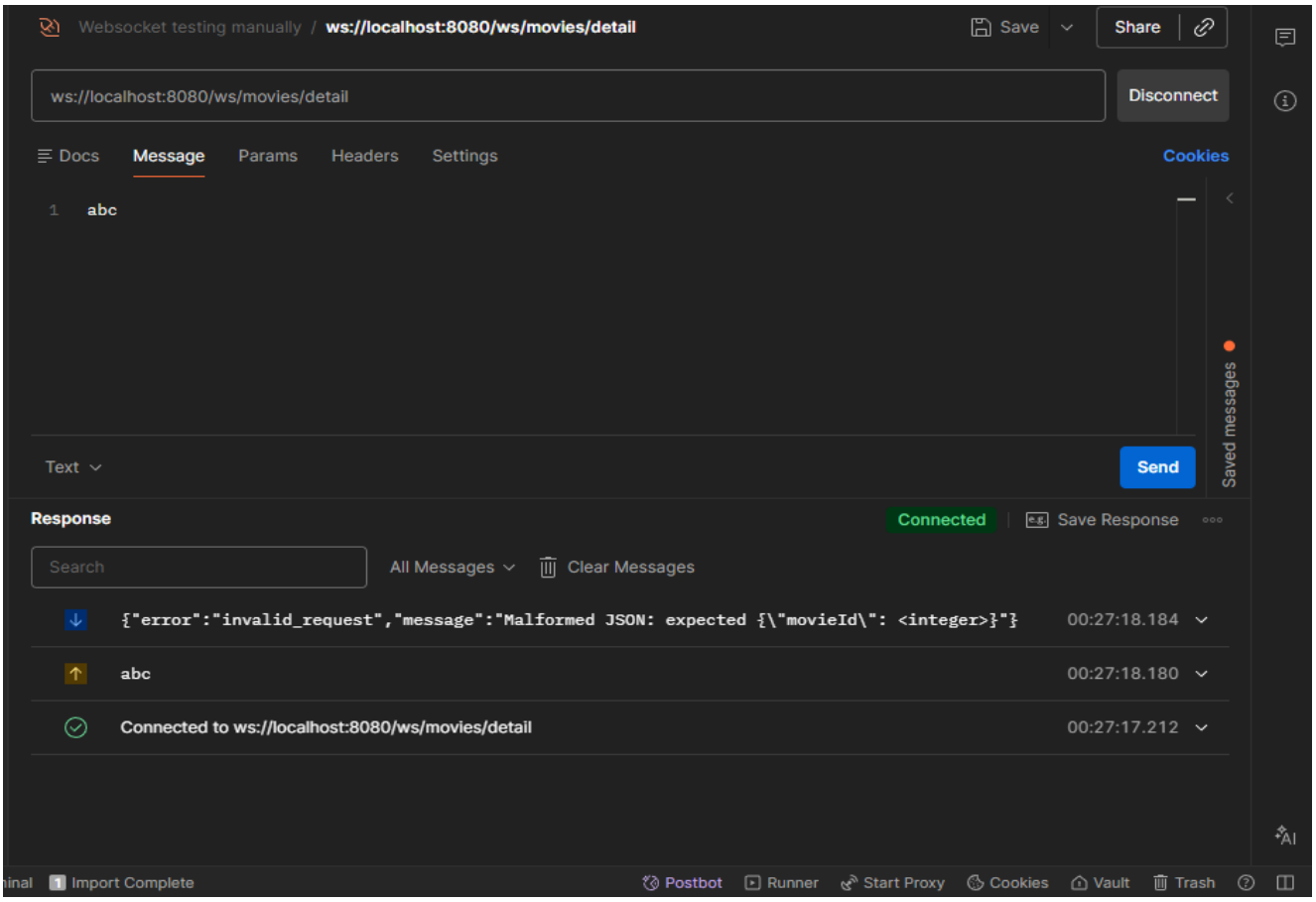
abc

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send another valid request on the same connection.

Result:



Test 3 — GetMovieDetail (not found)

Endpoint: `ws://localhost:8080/ws/movies/detail`

Request sent:

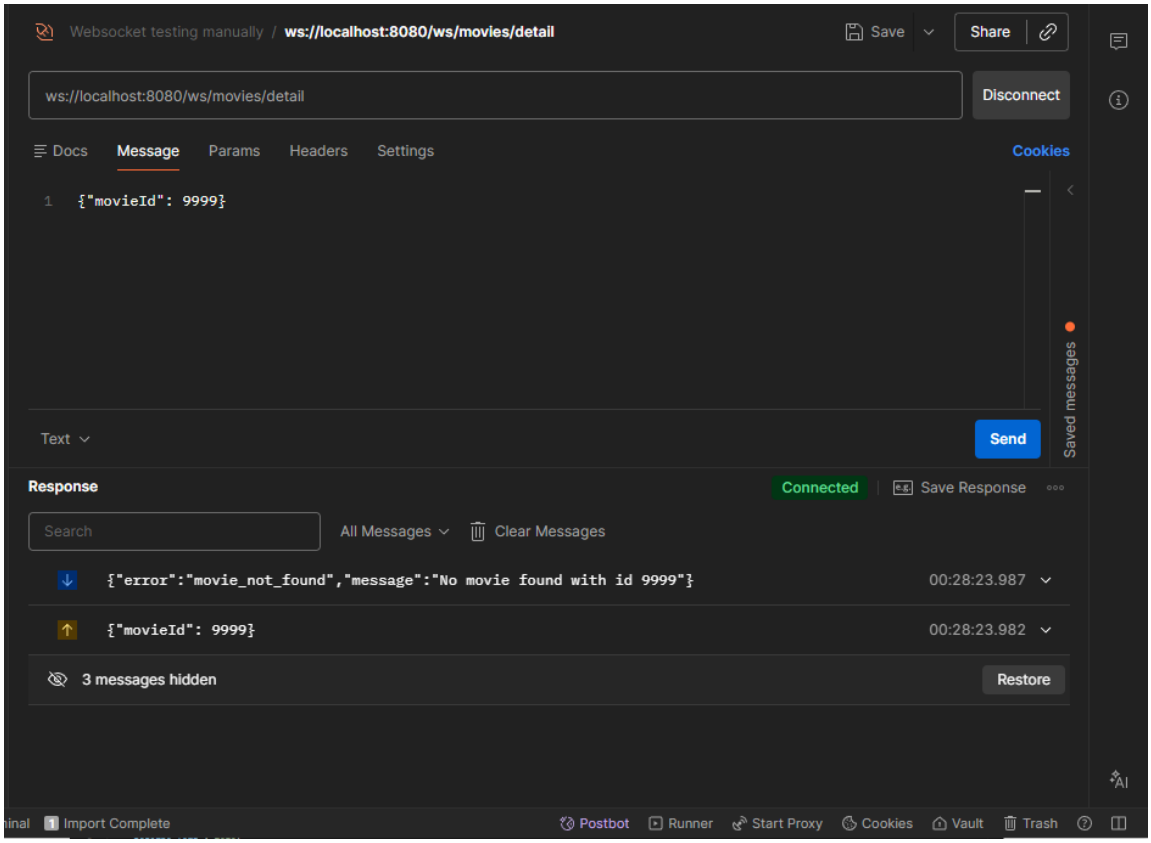
```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result:



Positive testing: Bidirectional Streaming operations

Test 4 — ReviewStream (subscribe and receive reviews)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent:

```
{"movieId": 1}
```

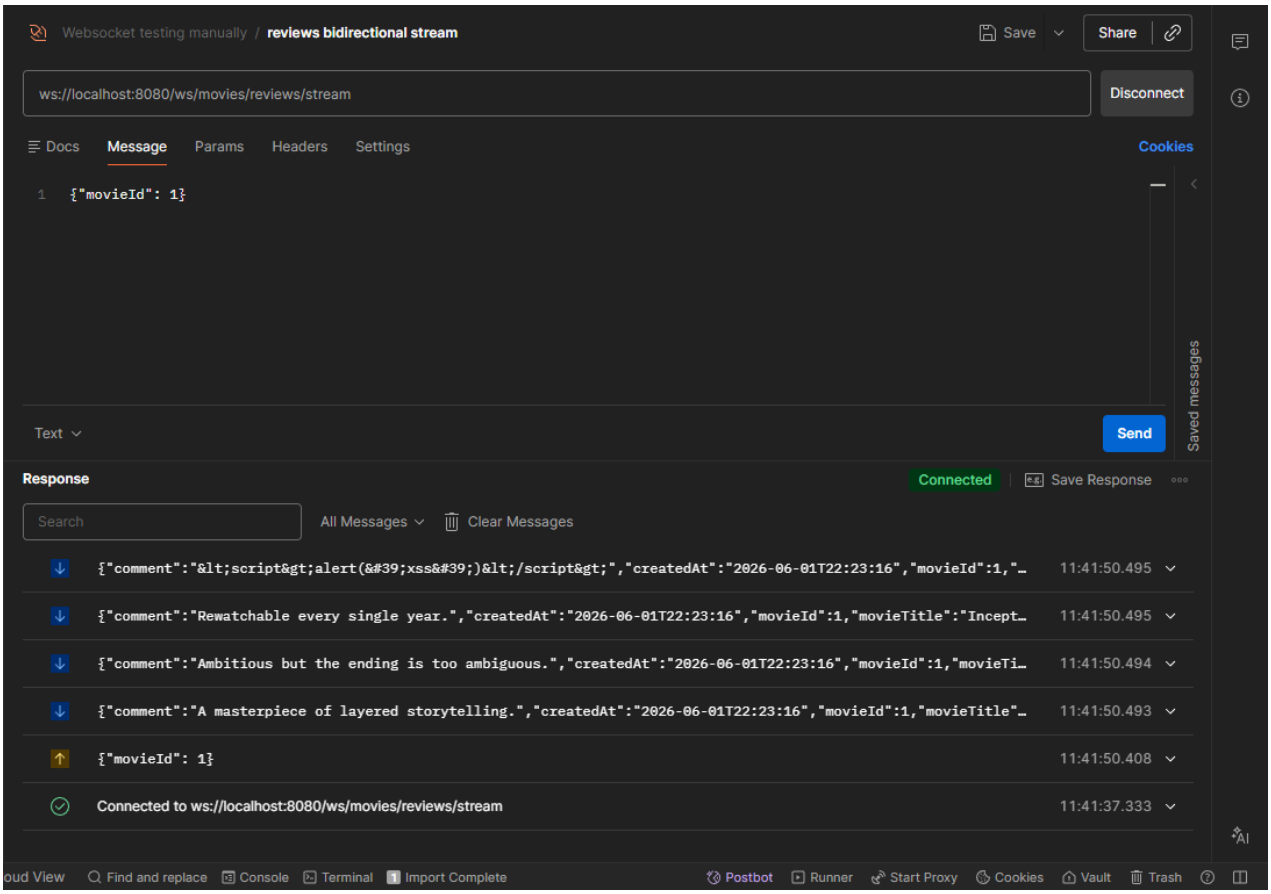
Expected response: The server immediately streams all existing reviews for movie 1 as individual `ReviewResponse` JSON frames. Each frame contains `reviewId`, `movieId`, `movieTitle`, `rating`, `comment`, and `createdAt`.

Movie 1 (Inception) has 4 reviews in the database (ids 101, 102, 103, 111), so we expect 4 frames. Example first frame:

```
{
  "reviewId": 101,
  "movieId": 1,
  "movieTitle": "Inception",
  "rating": 9,
  "comment": "A masterpiece of layered storytelling.",
  "createdAt": "..."
}
```

The connection stays open. The server continues polling every ~2 seconds for new reviews.

Result:



Test 5 — ReviewStream (add second subscription on same connection)

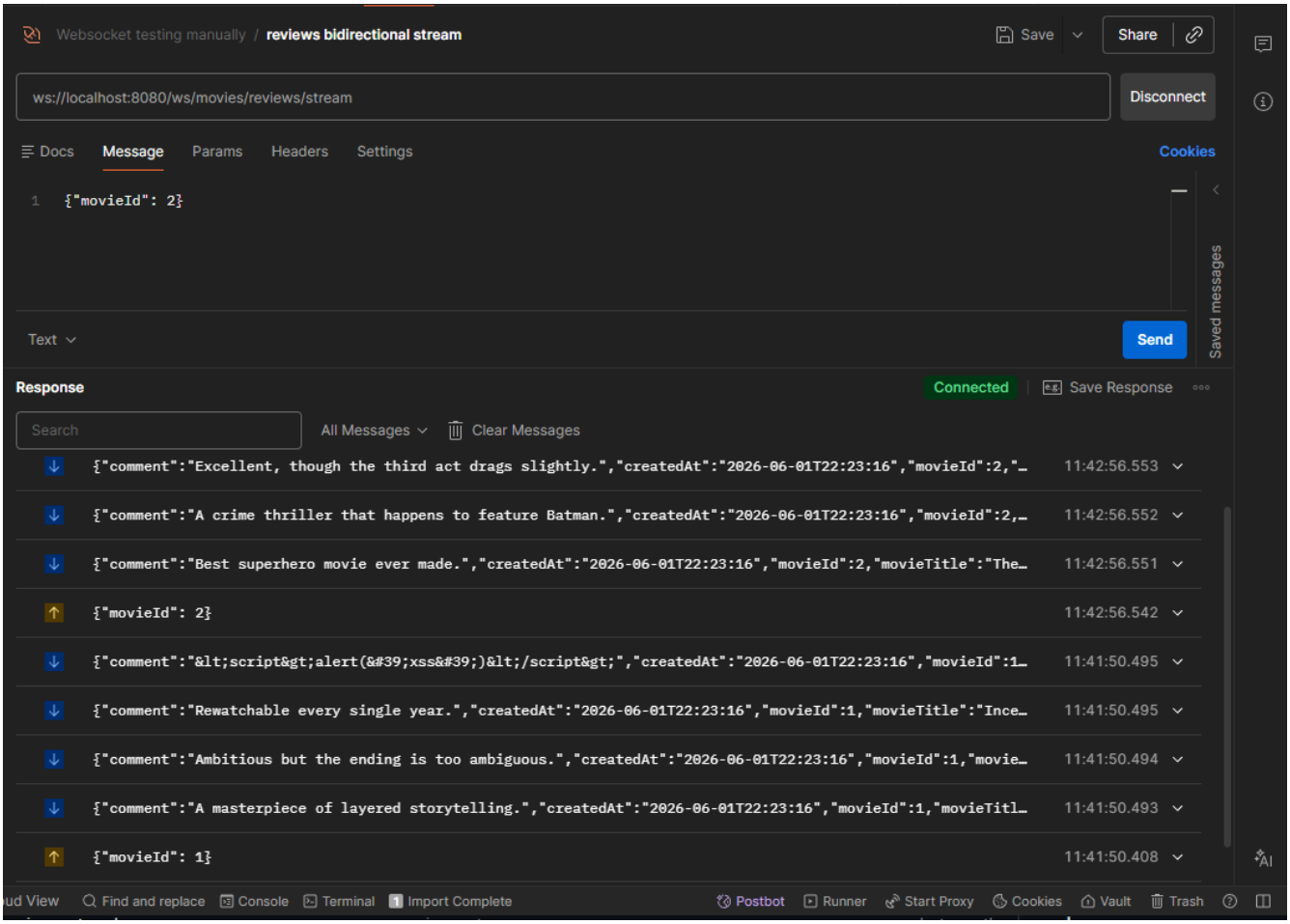
Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent (on same connection):

```
{"movieId": 2}
```

Expected response: The server streams all existing reviews for movie 2 (The Dark Knight) on the same connection. Movie 2 has 3 reviews in the database (ids 104, 105, 106), so we expect 3 additional frames. Both movie 1 and movie 2 reviews now flow on this connection.

Result:



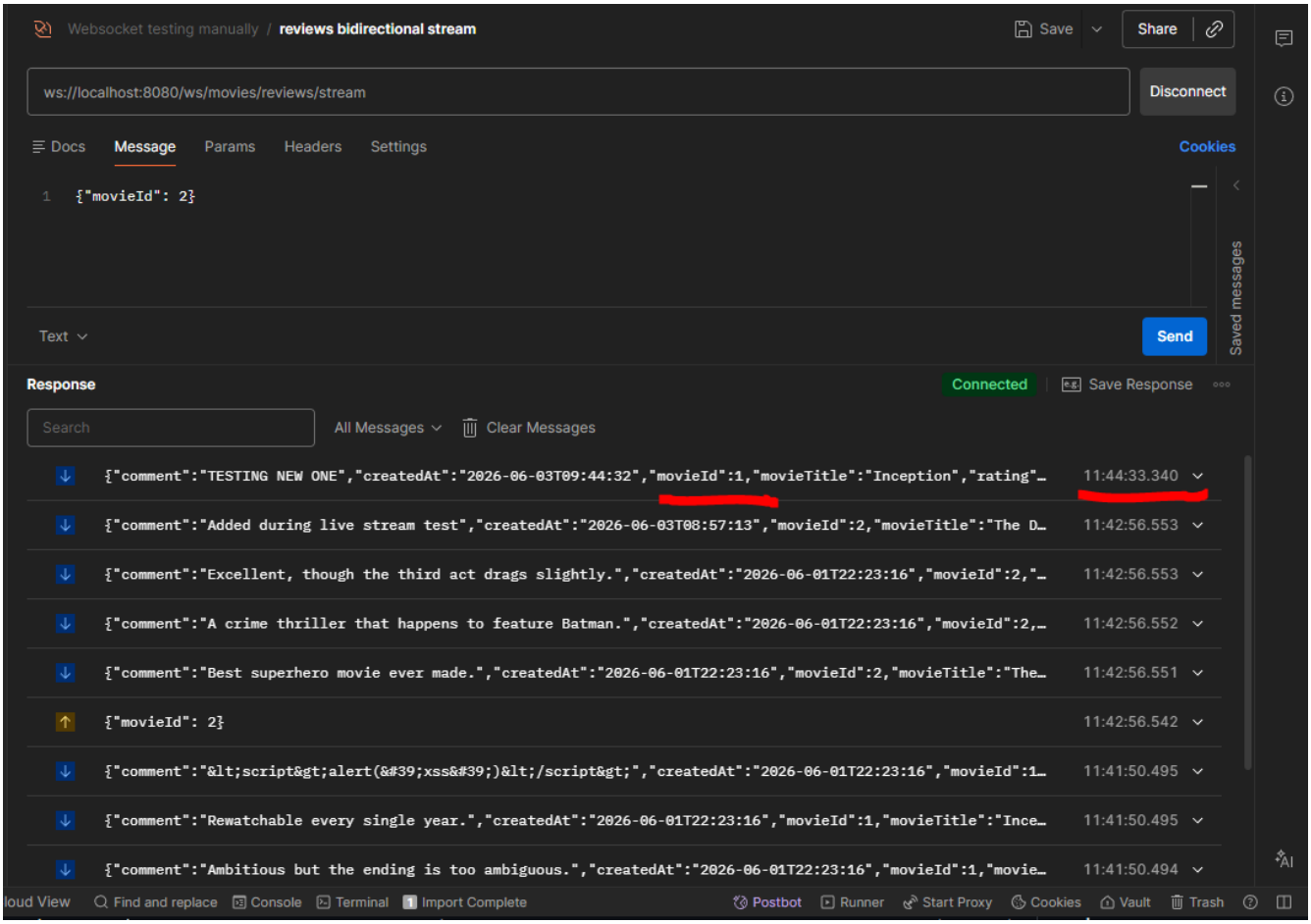
Test 6 — ReviewStream (live push — add review while subscribed)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

While the stream connection is open and subscribed to movie 1, a new review is added

Expected response: Within ~2 seconds, the newly added review appears as a `ReviewResponse` frame on the WebSocket connection — without the client sending any message.

Result:



Negative testing: Bidirectional Streaming operations

Test 7 — ReviewStream (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent:

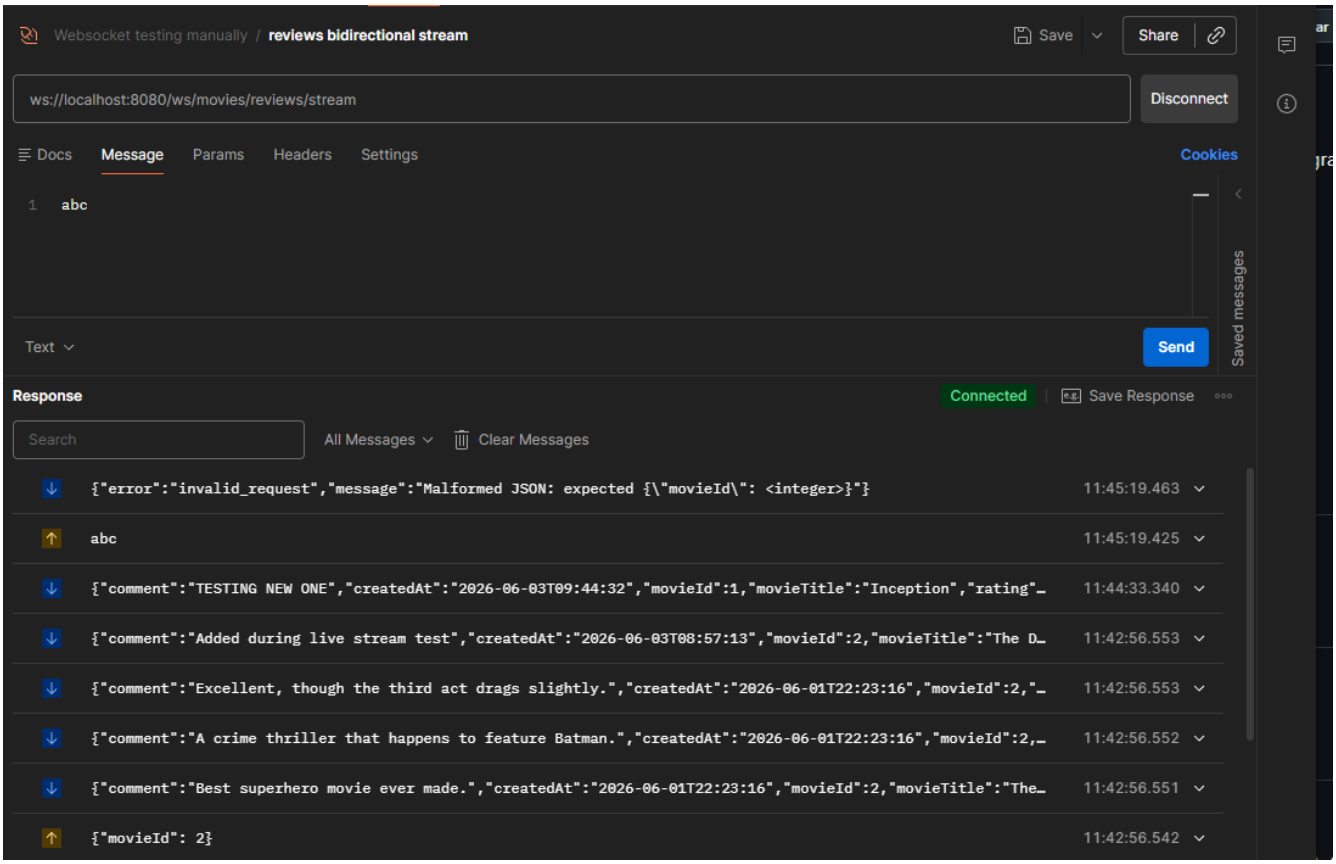
abc

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send a valid subscription on the same connection.

Result:



Test 8 — ReviewStream (not found)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent:

```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result: